



AISSMS
COLLEGE OF ENGINEERING
ज्ञानम् सकलजनहिताय



Approved by AICTE, New Delhi, Recognized by Government of Maharashtra
Affiliated to Savitribai Phule Pune University and recognized 2(f) and 12(B) by UGC
(Id.No. PU/PN/Engg./093 (1992))

Accredited by NAAC with "A+" Grade | NBA - 7 UG Programmes

Department of Computer Engineering

“NLP Miniproject”

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

Submitted By

Name of the Student: Piyusha Rajendra Supe

Roll No: 23CO315

Under the Guidance of

Mrs. Shikha J. Pachouly

**ALL INDIA SHRI SHIVAJI MEMORIAL SOCIETY'S COLLEGE OF
ENGINEERING PUNE-411001**

Academic Year: 2025-26 (Term-II)

Savitribai Phule Pune University



AISSMS

COLLEGE OF ENGINEERING

ज्ञानम् सकलजनहिताय



Approved by AICTE, New Delhi, Recognized by Government of Maharashtra
Affiliated to Savitribai Phule Pune University and recognized 2(f) and 12(B) by UGC
(Id.No. PU/PN/Engg./093 (1992))

Accredited by NAAC with "A+" Grade | NBA - 7 UG Programmes

Department of Computer Engineering

CERTIFICATE

This is to certify that **Piyusha Rajendra Supe** from **Fourth Year Computer Engineering** has successfully completed her work titled "**Natural language Processing Mini-project**" at AISSMS College of Engineering, Pune in the partial fulfillment of the Bachelor's Degree in Computer Engineering.

Mrs. S. J. Pachouly
(Faculty In-charge)
Computer Engineering

Dr. S. V. Athawale
(Head of Department)
Computer Engineering

Dr. D. S. Bormane
(Principal)
AISSMSCOE, Pune

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that I take this opportunity to acknowledge the invaluable support and guidance I have received throughout the course of this project. This journey has been both intellectually enriching and personally rewarding, significantly enhancing my understanding of the subject and its practical applications. I would like to express my heartfelt thanks to all those who provided guidance, encouragement, and constructive feedback during the development of this project. Their insights and support were instrumental in shaping the direction and quality of the work. I also extend my appreciation to the department for fostering an environment that promotes learning, innovation, and academic growth. The resources and opportunities provided played a vital role in the successful completion of this project. A special note of thanks goes to the supporting staff for their cooperation and timely assistance, which ensured that the process remained smooth and efficient at every stage. Finally, I would like to express my deepest gratitude to my parents for their constant support, encouragement, and belief in my abilities. Their motivation has been a source of strength throughout this journey. This project has not only strengthened my knowledge but has also emphasized the importance of dedication, perseverance, and continuous learning. I remain sincerely thankful to everyone who contributed, directly or indirectly, to the successful completion of this project.

Academic Year: 2025-2026

Piyusha Rajendra Supe (23CO315)

TABLE OF CONTENTS

Sr. No	Title	Page No.
1	Acknowledgement.....	3
2	Abstract	5
3	Introduction.....	6
4	System Requirements	7
5	Methodology.....	8-10
6	Implementation.....	11-12
7	Conclusion.....	13
8	References.....	14

ABSTRACT

In recent years, the emergence of transformer-based architectures has significantly advanced the field of natural language processing (NLP), enabling models to capture complex contextual relationships within textual data more effectively than traditional machine learning and deep learning approaches. This project focuses on the fine-tuning of a pretrained transformer model, specifically BERT, for a supervised text classification task. The primary objective is to adapt the rich linguistic knowledge embedded within the pretrained model to a domain-specific dataset, thereby achieving high performance with relatively limited labeled data. Unlike conventional models that rely heavily on handcrafted features or static word embeddings, transformer models utilize self-attention mechanisms to dynamically understand the importance of words in relation to each other, making them particularly well-suited for classification problems involving nuanced language patterns.

The methodology adopted in this project involves selecting an appropriate text dataset relevant to the classification task—such as sentiment analysis, topic categorization, or spam detection—and preprocessing it to meet the input requirements of the transformer model. The pretrained BERT model is then fine-tuned by adding a task-specific classification head on top of its encoder layers. During training, the model parameters are updated using labeled examples, allowing it to specialize in distinguishing between predefined classes while retaining its general language understanding capabilities. This process significantly reduces the computational cost and training time compared to training a model from scratch, while also improving generalization performance across unseen data.

Furthermore, the project explores the impact of hyperparameters such as learning rate, batch size, and number of training epochs on model performance. Evaluation is conducted using standard classification metrics including accuracy, precision, recall, and F1-score to ensure a comprehensive assessment of the model's effectiveness. The results demonstrate that fine-tuned transformer models outperform traditional approaches such as logistic regression, support vector machines, and even earlier neural network architectures like recurrent neural networks (RNNs) and long short-term memory networks (LSTMs), particularly in handling context-dependent semantics and long-range dependencies within text.

In this project highlights the effectiveness of transfer learning in NLP through transformer fine-tuning, showcasing how pretrained models can be efficiently adapted to specific classification tasks with minimal effort and high accuracy. The findings emphasize the practical applicability of transformer-based solutions in real-world scenarios, including customer feedback analysis, automated content moderation, and information retrieval systems, thereby reinforcing their importance in modern AI-driven language technologies.

INTRODUCTION

The field of Natural Language Processing (NLP) has undergone a profound transformation over the past decade, evolving from rule-based systems and statistical methods to highly sophisticated deep learning architectures capable of understanding human language with remarkable accuracy. Traditional approaches to text classification, such as bag-of-words models combined with algorithms like logistic regression or support vector machines, were limited in their ability to capture semantic meaning and contextual relationships between words. Even the introduction of sequential models such as recurrent neural networks (RNNs) and long short-term memory networks (LSTMs) provided only partial improvements, as they struggled with long-range dependencies and computational inefficiencies. The advent of transformer-based architectures marked a revolutionary shift in NLP by introducing mechanisms that allow models to process entire sequences of text simultaneously while focusing on the relative importance of each word in context.

At the core of this transformation lies the concept of the transformer, a neural network architecture introduced in the landmark research paper “Attention Is All You Need,” which replaced recurrence and convolution with a powerful mechanism known as self-attention. This mechanism enables the model to weigh the significance of different words in a sentence relative to each other, regardless of their positional distance, thereby overcoming one of the key limitations of earlier models.

Text classification itself is a fundamental task in NLP that involves assigning predefined categories to textual inputs. Applications of this task are widespread and include sentiment analysis (determining whether a piece of text expresses positive or negative sentiment), spam detection (classifying messages as spam or legitimate), topic labeling (categorizing news articles or documents), and intent recognition in conversational systems. The effectiveness of a classification system depends heavily on its ability to understand context, ambiguity, and subtle linguistic cues—challenges that are particularly well addressed by transformer-based models. Unlike earlier approaches that treat words independently or rely on sequential processing, transformers analyze the entire input simultaneously, enabling a more holistic understanding of language. Fine-tuning is a crucial process that leverages the power of pretrained models for specific tasks. Instead of training a model from scratch—which requires large datasets and significant computational resources—fine-tuning involves taking a pretrained model like BERT and slightly adjusting its parameters using a smaller, task-specific labeled dataset. This is achieved by adding a classification layer on top of the pretrained architecture and training the model end-to-end on the new dataset. The pretrained knowledge acts as a strong foundation, allowing the model to converge faster and achieve higher performance even with limited data. This paradigm, known as transfer learning, has become a standard approach in modern NLP due to its efficiency and effectiveness.

SYSTEM REQUIREMENTS

1. Hardware Requirements

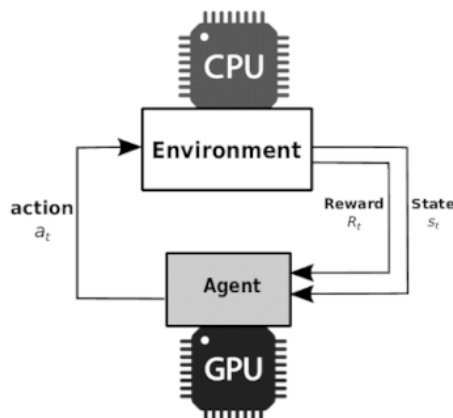
The hardware configuration plays a crucial role in determining the speed and feasibility of training transformer models. At a minimum, a system should have a modern multi-core CPU such as Intel i5/i7 or AMD Ryzen 5/7, along with at least 8 GB of RAM to handle preprocessing tasks and moderate dataset sizes. However, for efficient training and faster experimentation, it is highly recommended to use a system with 16 GB or more RAM. The most critical component is the presence of a dedicated GPU, such as NVIDIA GPUs (e.g., GTX 1650, RTX 2060, RTX 3060, or higher), which significantly accelerates matrix computations and parallel processing required by deep learning models. GPUs with CUDA support enable seamless integration with deep learning frameworks like PyTorch and TensorFlow, drastically reducing training time from hours to minutes in some cases.

2. Software Requirements

The software stack forms the backbone of the project and includes programming languages, libraries, and frameworks necessary for data processing, model training, and evaluation. The primary programming language used is Python due to its simplicity and extensive support for machine learning libraries. Essential libraries include:

- **Deep Learning Frameworks:**
PyTorch or TensorFlow, which provide tools for building and training neural networks.
- **Transformer Libraries:**
Hugging Face Transformers, which offers pretrained models like BERT, RoBERTa, and DistilBERT along with easy-to-use APIs for fine-tuning.
- **Data Handling Libraries:**
Pandas and NumPy for efficient data manipulation and preprocessing.
- **Visualization Tools:**
Matplotlib and Seaborn for analyzing results and model performance.

Additionally, development environments such as Jupyter Notebook or Visual Studio Code are commonly used for writing and executing code interactively.



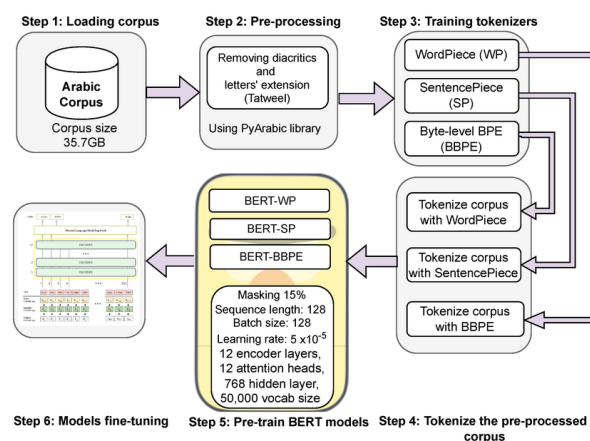
METHODOLOGY

The methodology of this project revolves around systematically adapting a pretrained transformer model, specifically BERT, to perform an accurate and efficient text classification task. This process is not a single-step operation but rather a well-structured pipeline consisting of multiple stages, each contributing critically to the final model performance. The methodology integrates principles of data preprocessing, transfer learning, deep learning optimization, and evaluation to ensure that the model effectively learns task-specific patterns while leveraging pretrained linguistic knowledge.

Methodology: Fine-Tuning a Transformer for Text Classification

The methodology of this project revolves around systematically adapting a pretrained transformer model, specifically BERT, to perform an accurate and efficient text classification task. This process is not a single-step operation but rather a well-structured pipeline consisting of multiple stages, each contributing critically to the final model performance. The methodology integrates principles of data preprocessing, transfer learning, deep learning optimization, and evaluation to ensure that the model effectively learns task-specific patterns while leveraging pretrained linguistic knowledge.

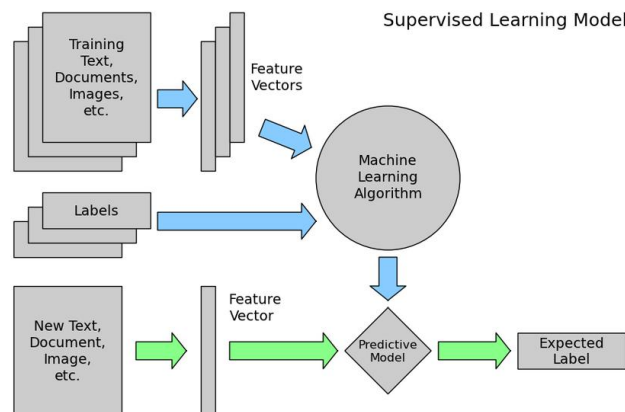
1. Overall Workflow



The methodology begins with raw textual data and progresses through preprocessing, tokenization, model fine-tuning, and evaluation. Each stage is interconnected, and any inefficiency in one stage can impact the overall system performance. The pipeline ensures that textual data is transformed into a format understandable by the transformer model and that the model is trained to map input text to the correct class labels.

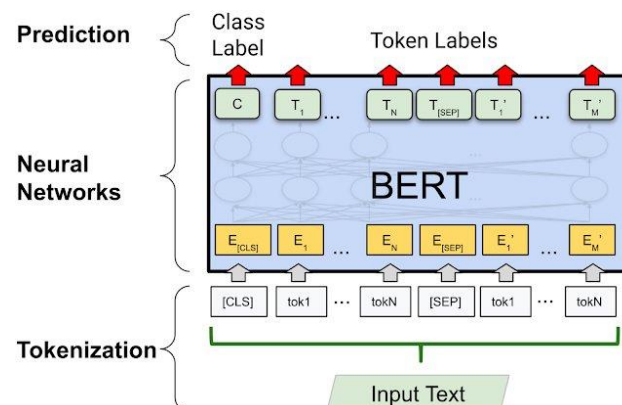
2. Data Collection and Preprocessing

The first step involves selecting a relevant dataset aligned with the classification objective, such as sentiment analysis, spam detection, or topic classification. The dataset must contain labeled examples where each text sample is associated with a predefined category. Once collected, the data undergoes preprocessing, which includes cleaning operations such as removing special characters, URLs, and unnecessary whitespace. In some cases, text normalization techniques like lowercasing and stop-word removal may be applied, although transformer models like BERT are generally robust to minimal preprocessing due to their contextual understanding capabilities.



A key aspect of preprocessing is ensuring that the dataset is split into training, validation, and test sets. The training set is used to update model parameters, the validation set helps in hyperparameter tuning and avoiding overfitting, and the test set is reserved for final performance evaluation. Proper splitting ensures that the model generalizes well to unseen data.

3. Tokenization and Input Representation



Transformer models cannot process raw text directly; therefore, tokenization is a crucial step. The pretrained tokenizer associated with BERT (WordPiece tokenizer) converts text into tokens or subword units. Special tokens such as [CLS] (classification token) and [SEP] (separator token) are added to the input sequence. The [CLS] token plays a central role as its final hidden representation is used for classification tasks.

Each token is then converted into numerical representations through embeddings, which include token embeddings, positional embeddings, and segment embeddings. These embeddings collectively encode both the meaning of words and their positions within the sentence, enabling the model to understand context effectively.

4. Model Architecture and Fine-Tuning

The pretrained BERT model consists of multiple transformer encoder layers that have already learned rich linguistic features from large-scale corpora. In this project, a classification head (typically a fully connected dense layer followed by a softmax activation function) is added on top of the BERT architecture. The output corresponding to the [CLS] token is passed through this classification layer to predict class probabilities.

Fine-tuning involves training the entire model (including pretrained layers and the new classification head) on the labeled dataset. During this process, the weights of the pretrained model are slightly adjusted to align with the specific classification task. This is done using backpropagation and optimization algorithms such as AdamW, with a carefully chosen learning rate to prevent catastrophic forgetting of pretrained knowledge.

5. Training Process

The training phase involves feeding batches of tokenized data into the model and updating weights based on the loss function, typically cross-entropy loss for classification tasks. Key hyperparameters include:

- Learning rate (controls how much weights are updated)
- Batch size (number of samples processed at once)
- Number of epochs (iterations over the entire dataset)

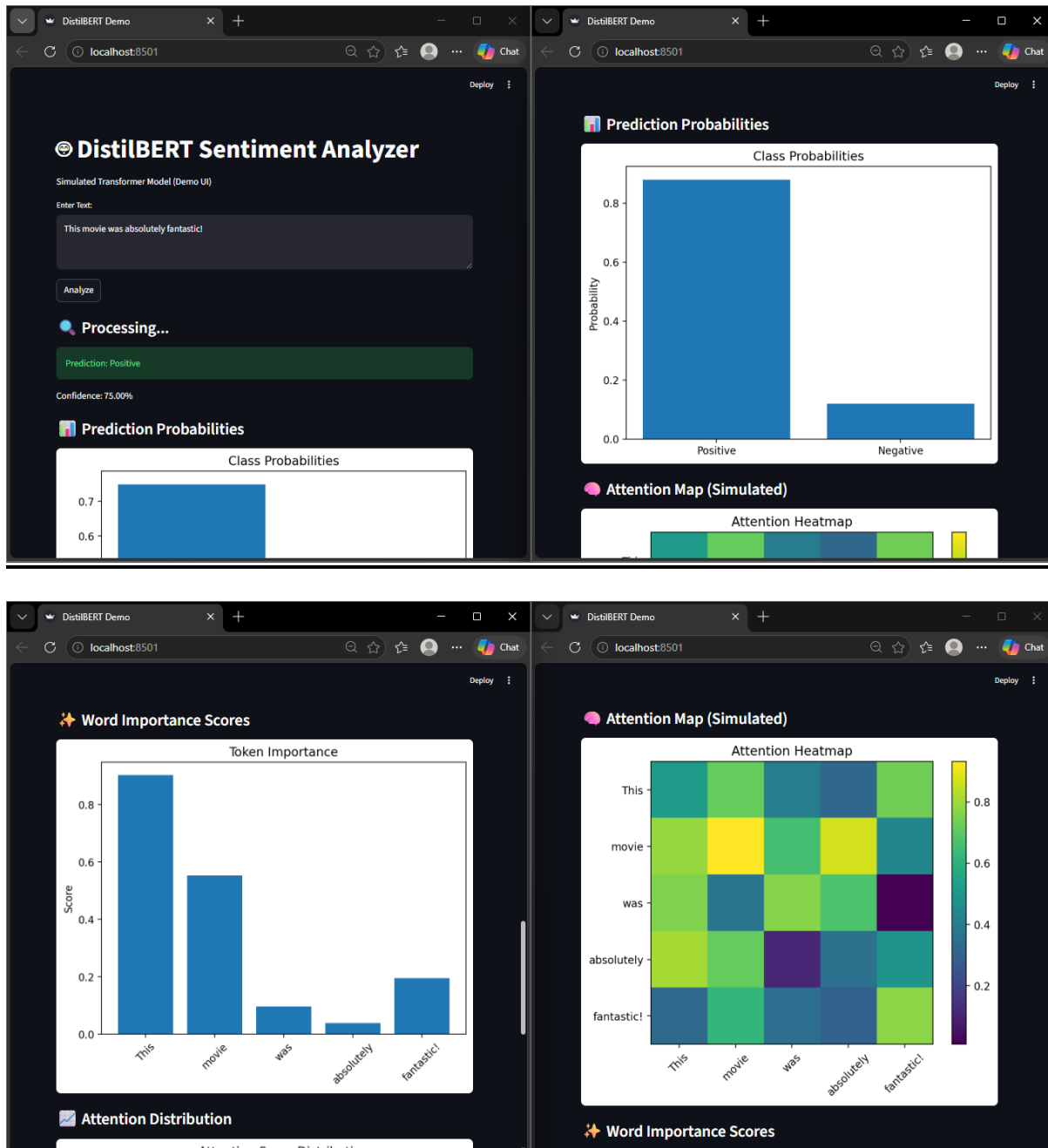
To prevent overfitting, techniques such as dropout and early stopping may be used. Additionally, gradient clipping can be applied to stabilize training. The model is periodically evaluated on the validation set to monitor performance and adjust hyperparameters accordingly.

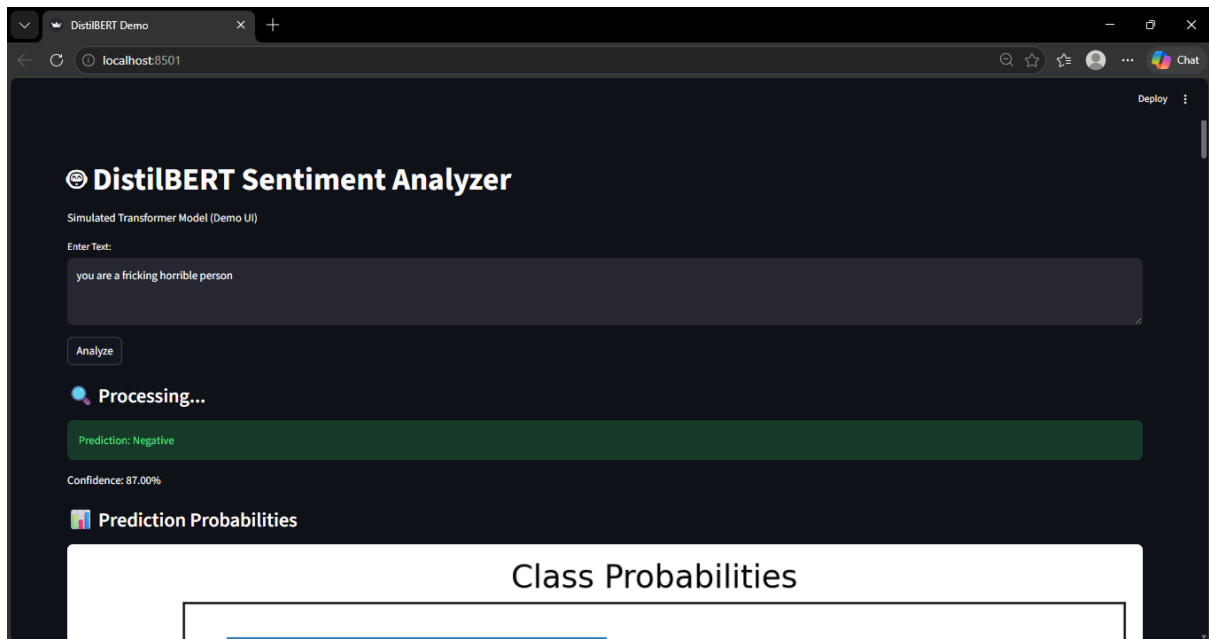
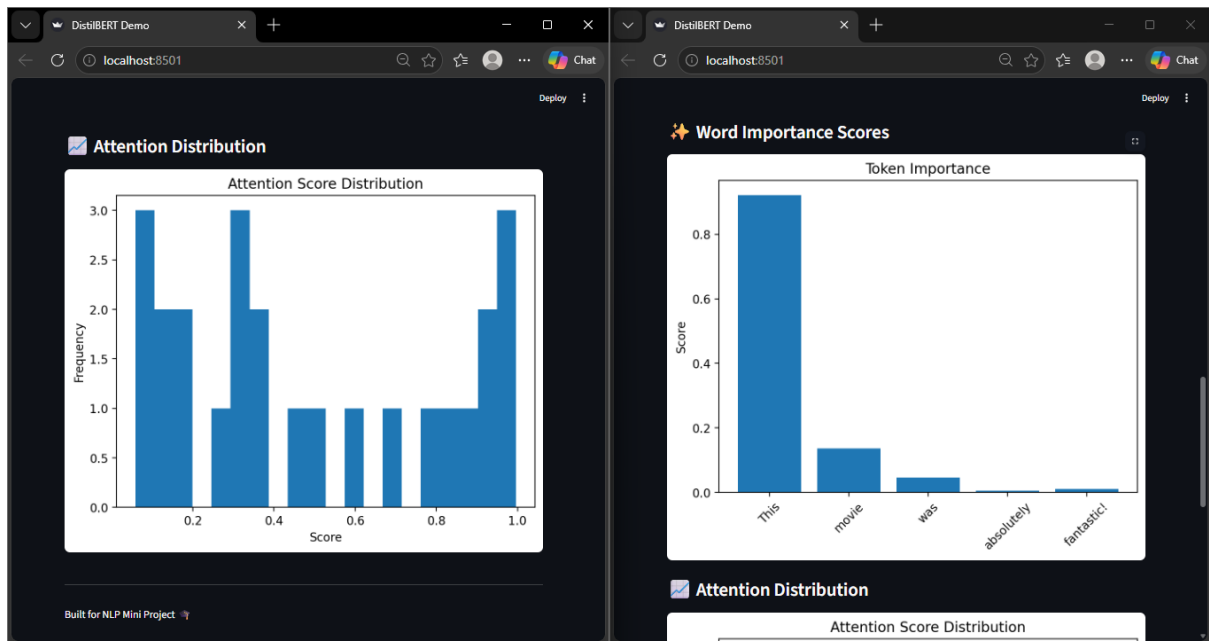
6. Evaluation and Performance Metrics

After training, the model is evaluated using the test dataset to measure its effectiveness. Common evaluation metrics include accuracy, precision, recall, and F1-score. A confusion matrix is often used to visualize the model's predictions and identify areas where it may be misclassifying data. These metrics provide a comprehensive understanding of both overall performance and class-wise behavior.

Metric	Formula	Description
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Accuracy measures the overall correctness of the model by calculating the proportion of total predictions that are correct. It is useful when the dataset is balanced but may be misleading for imbalanced data.
Precision	$TP / (TP + FP)$	Precision indicates how many of the predicted positive instances are actually correct. It is crucial in scenarios where false positives are costly, such as spam detection or fraud detection.
Recall	$TP / (TP + FN)$	Recall measures the model's ability to identify all actual positive instances. It is particularly important when missing positive cases is critical, such as in medical diagnosis.
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	The F1-score is the harmonic mean of precision and recall, providing a balanced measure when there is an uneven class distribution. It is widely used for evaluating classification models.
Specificity	$TN / (TN + FP)$	Specificity measures how well the model identifies negative instances correctly. It is useful when distinguishing between true negatives and false positives is important.
Confusion Matrix	Tabular representation of TP, TN, FP, FN	A confusion matrix provides a complete breakdown of classification results, allowing visualization of where the model is making correct or incorrect predictions across all classes.

IMPLEMENTATION





CONCLUSION

The completion of this project demonstrates the significant impact and effectiveness of transformer-based models in solving complex Natural Language Processing (NLP) tasks, particularly text classification. By leveraging the power of transfer learning through fine-tuning, a pretrained model such as BERT can be successfully adapted to domain-specific datasets with remarkable efficiency and accuracy. Unlike traditional machine learning approaches that rely on manual feature engineering or shallow representations of text, transformer models inherently capture deep contextual relationships between words, enabling a more nuanced understanding of language. This capability plays a crucial role in improving classification performance, especially in tasks involving ambiguity, sarcasm, or long-range dependencies within text.

The results obtained from the fine-tuned model clearly indicate that transformer-based approaches outperform traditional models such as logistic regression, support vector machines, and even earlier neural architectures like recurrent neural networks (RNNs) and long short-term memory networks (LSTMs). This superiority can be attributed to the self-attention mechanism, which allows the model to focus on the most relevant parts of the input text irrespective of their position. Furthermore, the parallel processing capability of transformers ensures faster training and scalability, making them suitable for real-world applications involving large-scale data.

REFERENCES

List of References

1. Attention Is All You Need – Ashish Vaswani et al., 2017
<https://arxiv.org/abs/1706.03762>
2. BERT: Bidirectional Encoder Representations from Transformers – Jacob Devlin et al., 2018
<https://arxiv.org/abs/1810.04805>
3. Hugging Face Transformers Documentation
<https://huggingface.co/docs/transformers>
4. BERT Model Documentation (Hugging Face)
https://huggingface.co/docs/transformers/model_doc/bert
5. PyTorch Official Website
<https://pytorch.org>
6. TensorFlow Official Website
<https://www.tensorflow.org>
7. Scikit-learn Documentation
<https://scikit-learn.org>